

Notas de Programación Competitiva

Milena Pellegrino

2025

Prefacio

Tenia este espacio al pedo y no sabia que escribir, le puse prefacio para que parezca profesional pero dice huevadas, despues se me ocurrio escribir un miini template que no tenga tantas cosas como mi template pero me quedaba grande la hoja y ademas ya habia gastado tiempo en escribir este hermoso prefacio, asi que simplemente deje esto, y puse el template abajo.

```
#include<bits/stdc++.h>
using namespace std;

#define fore(i,a,b) for(ll i=(a);i<(b);i++)
#define all(x) (x).begin(),(x).end()
#define pb push_back
#define fst first
#define snd second
#define sz(x) ((int)x.size())

#define FIN ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);

using ll = long long;
typedef vector<ll> vi;

int main(){
    FIN;

    return 0;
}
```

Listing 1: Mi mini template bro

Me quedo este espacio tambien libre y quise escribir esto para parecer toda una profesional, pero no se me ocurrio algo interesante, asi que, que le dice el 1 al 10? para ser como yo tienes que ser sincero.

Índice general

1. Introducción	1
1.1. Sobre las notas	1
1.2. Cosas de C++	1
1.2.1. Tamaño de los tipos de datos	1
1.2.2. Lectura de líneas con espacios en C++	1
1.2.3. Conversiones	1
String a Numero y de Numero a String	1
Conversiones de char a número posicionales	2
De mayuscula a minuscula y visceversa	2
Conversiones a binarias y visceversas	2
1.3. Cosas comunes en programación competitiva	3
1.3.1. Notación BIG O	3
1.3.2. Resize() en c++	4
2. Libreria Estandar C++ (STD)	5
2.1. Queue (cola)	5
2.2. Stack (pila)	5
2.3. Set (conjunto)	6
2.4. Representación Binaria en C++	7
2.4.1. Mostrar un número en binario	7
2.4.2. Obtener la representación binaria como cadena	7
2.4.3. Conversión binaria manual	7
3. Búsquedas	9
3.1. Búsqueda binaria	9
3.1.1. Función lower_bound	9
3.2. Problema de ejemplo	9
3.2.1. Descripción	9
3.2.2. Solución	9
3.2.3. Código de Solución	10
3.3. Uso de lower_bound para obtener índices	10
3.3.1. Obtención del índice	10
3.3.2. Problema	11
3.4. Búsqueda binaria en la respuesta	12
3.4.1. Caso general	12
3.4.2. Ejemplo 1: Problema de Sumas	13
3.4.3. Ejemplo 2: Problema de Porcentajes	13
3.4.4. Ejemplo 3: Problema de Recursos	14

3.4.5.	Consideraciones Importantes	15
3.4.6.	Complejidad	15
4.	Matemática	17
4.1.	Operaciones basicas	17
4.1.1.	Obtener π	17
4.1.2.	Obtener la raiz cuadrada	17
4.2.	Reglas de Divisibilidad	18
4.2.1.	Reglas de Divisibilidad Básicas	18
4.2.2.	Reglas de Divisibilidad Menos Comunes	18
4.2.3.	Reglas de Divisibilidad para Números Primos Mayores	19
4.2.4.	Métodos Generales para Verificar Divisibilidad	19
4.2.5.	Generación de divisores	19
4.3.	Números Primos	19
4.3.1.	Chequear si un número es primo	19
4.3.2.	Criba de Eratóstenes para factorizar números	20
4.4.	Permutaciones	21
4.4.1.	Computar las permutaciones	21
4.5.	Geometría	21
4.5.1.	Distancia entre dos puntos	21
4.5.2.	Triángulos	22
	Ley de Cosenos	22
	Ley de Senos	22
5.	Programacion Dinamica (dp)	23
5.1.	De una función recursiva por partes a código con DP	23
5.2.	Ejemplos clásicos	24
5.2.1.	A - Frog 1	24
5.2.2.	C - Vacation	25
5.2.3.	D - Knapsack 0/1	25
5.2.4.	G - Longest Path in DAG	26
5.3.	Resumen	26
6.	Grafos	27
6.1.	Representaciones comunes	27
6.2.	Recorridos clásicos	27
6.2.1.	DFS (Depth-First Search)	27
6.2.2.	BFS (Breadth-First Search)	28
6.3.	Componentes conexas	28
6.4.	Dijkstra (camino mínimos)	28
6.5.	Bellman-Ford	29
6.6.	Problemas clásicos	30
6.7.	¿Cómo verificar si un grafo es una línea (path)?	30
6.8.	Detectar ciclos en grafo no dirigido	31
6.9.	Detectar ciclos en grafo dirigido	32
6.10.	Verificar si hay ciclo impar	33
6.11.	Verificar si el grafo es bipartito	33
6.12.	Distancia mínima desde un nodo	34
6.13.	Verificar si el grafo es un árbol	35

6.14. Número de hojas en un árbol	36
6.15. Reconstrucción de Caminos (BFS)	36
6.16. Orden Topológico	37
7. Strings	41
7.1. Permutaciones de string	41
8. Estructuras de Datos	43
8.1. Union Find / Disjoint Set Union (DSU)	43
8.2. Fenwick Tree (BIT)	44
9. Arrays / Subarrays	47
9.1. Kadane	47
9.1.1. Kadane con restricción (el subarreglo debe incluir k)	47

Capítulo 1

Introducción

1.1. Sobre las notas

Estas notas explican recordatorios y cosas basicas en programación competitiva. Fueron escritas con la idea de usarla como notebook en competencias.

1.2. Cosas de C++

1.2.1. Tamaño de los tipos de datos

- **Tamaño de un long long:** Es un número de 64 bits, 2^{64} o aproximadamente 10^{19} .

1.2.2. Lectura de líneas con espacios en C++

En C++, para leer una línea completa que puede contener espacios (como una oración o una entrada con varios números separados), se utiliza:

```
getline(std::cin, s);
```

Sin embargo, si antes se usó

```
std::cin >> variable;
```

entonces `getline()` puede leer una línea vacía. Para evitarlo, se debe limpiar el buffer con:

```
std::cin.ignore();
```

```
cin.ignore();
```

1.2.3. Conversiones

String a Numero y de Numero a String

- De string a int: `int x = stoi("123");`
- De string a long long: `ll y = stoll("9876543210");`
- De numero a string: `string s = to_string(x);`

Conversiones de char a número posicionales

De char numeral a número: Si el char representa un dígito (por ejemplo, '0' a '9'), se puede obtener el valor numérico restando el carácter '0':

```
11 x = s[0] - '0'; // convierte '0' a 0, '5' a 5, etc.
```

De char minúscula a número ($a \rightarrow 1, b \rightarrow 2, c \rightarrow 3, \dots$): Dado un carácter en minúscula $c \in \{a, b, \dots, z\}$, se puede obtener su posición en el alfabeto con:

$$\text{posición}(c) = c - 'a' + 1$$

Ejemplos:

```
int pos_b = 'b' - 'a' + 1; // pos_b == 2
int pos_z = 'z' - 'a' + 1; // pos_z == 26
```

De char mayúscula a número ($A \rightarrow 1, B \rightarrow 2, C \rightarrow 3, \dots$): Para caracteres en mayúscula $C \in \{A, B, \dots, Z\}$, se usa la misma lógica:

$$\text{posición}(C) = C - 'A' + 1$$

Ejemplos:

```
int pos_B = 'B' - 'A' + 1; // pos_B == 2
int pos_Z = 'Z' - 'A' + 1; // pos_Z == 26
```

De mayuscula a minuscula y viceversa

- Convertir un carácter a mayúscula:

```
char c = 'b';
c = toupper(c); // c ahora es 'B'
```

- Convertir un carácter a minúscula:

```
char c = 'G';
c = tolower(c); // c ahora es 'g'
```

Conversiones a binarias y viceversas

Convertir número a string binario: Para convertir un número entero a su representación binaria en forma de string, se puede usar la función:

```
11 num = 13;
string binary = bitset<64>(num).to_string(); // "0000...1101"

// 0 sin ceros a la izquierda:
binary = bitset<64>(num).to_string();
binary.erase(0, binary.find('1')); // remueve ceros iniciales
```

Convertir string binario a número: Para convertir un string que representa un número binario a su valor decimal, se usa `stoll()` con base 2:

```
string binary = "1101";
ll num = stoll(binary, nullptr, 2); // num == 13

// Otros ejemplos:
ll a = stoll("0101", nullptr, 2); // a == 5
ll b = stoll("11111111", nullptr, 2); // b == 255
```

Nota: El parámetro `nullptr` indica que no se necesita guardar la posición del primer carácter no convertido, y el 2 especifica que se usa base binaria.

Ordenar partes (subrangos) de un vector

En C++, se puede ordenar una parte de un vector (es decir, un subrango) utilizando la función `sort` de la STL, especificando iteradores de inicio y fin.

Esto es especialmente útil cuando se necesita dividir un conjunto en mitades o grupos, por ejemplo, para aplicar diferentes criterios de ordenamiento en cada parte.

```
// Supongamos que tenemos un vector de arreglos de 3 enteros:
vector<array<ll, 3>> v(n);

// Ordenamos solo la primera mitad del vector por el segundo campo (índice)
sort(v.begin(), v.begin() + n/2, [](const array<ll, 3> &a, const array<ll, 3> &b) {
    return a[1] < b[1];
});

// Ordenamos la segunda mitad también por el segundo campo:
sort(v.begin() + n/2, v.end(), [](const array<ll, 3> &a, const array<ll, 3> &b) {
    return a[1] < b[1];
});
```

Aplicación: esta técnica aparece frecuentemente en problemas donde hay que emparejar elementos de dos mitades, o donde diferentes partes del conjunto deben ser tratadas con diferentes estrategias.

1.3. Cosas comunes en programación competitiva

1.3.1. Notación BIG O

Tamaño del input: N	Peor complejidad aceptada
$N \leq 22$ ($\approx 2 \cdot 10^1$)	$O(2^N)$
$N \leq 100$ (10^2)	$O(N^3)$
$N \leq 1000$ (10^3)	$O(N^2)$
$N \leq 1,000,000$ (10^6)	$O(N \log N)$
$N \leq 1,000,000,000$ (10^9)	$O(N), O(\log N), O(1)$

Cuadro 1.1: Relación entre tamaño de input y la peor complejidad aceptada.

1.3.2. Resize() en c++

La función `std::vector::resize()` ajusta el número de elementos en un vector. Su acción depende del tamaño deseado:

- **Si el nuevo tamaño es mayor que el actual:** Se añaden nuevos elementos al final. Opcionalmente, se puede especificar un valor de inicialización: `mi_vector.resize(10, -1)`.
- **Si el nuevo tamaño es menor que el actual:** Se eliminan los elementos del final del vector hasta alcanzar el tamaño especificado.

```
int main() {  
    int n; cin>>n;  
    vector<int> numeros;  
    numeros.resize(n); // Pre-asigna memoria para n elementos  
    for (int i = 0; i < n; ++i) {  
        cin >> numeros[i]; // Acceso directo por \'indice  
    }  
    return 0;  
}
```

Listing 1.1: Uso eficiente de resize

Capítulo 2

Librería Estandar C++ (STD)

2.1. Queue (cola)

Las colas son estructuras first-in-first-out (FIFO).

Complejidad de todas las operaciones: $O(1)$

```
// Crear una queue:
queue<int> cola;

// Insertar elemento al FINAL de la cola:
cola.push(1)

// Ver cuantos elementos tiene:
cola.size()

// Eliminar el primer elemento:
cola.pop();

// VER cual es el primer elemento:
cola.front()

// Ver si la cola esta vacia:
if (cola.empty())
```

Listing 2.1: Funcionamiento de la queue

2.2. Stack (pila)

Las pilas son estructuras last-in-first-out (LIFO).

Complejidad de todas las operaciones: $O(1)$

```
// Crear un stack:
stack<int> pila;

// Insertar elemento al PRINCIPIO del stack:
pila.push(1);

// Ver cuantos elementos tiene:
pila.size()

// Eliminar el primer elemento:
pila.pop();
```

```
// VER cual es el primer elemento:
pila.top()

// Ver si la cola esta vacia:
if (pila.empty())
```

Listing 2.2: Funcionamiento del stack

2.3. Set (conjunto)

Almacena valores unicos de un determinado tipo, ordenados automaticamente.

```
// Crear un set:
set<int> s;

// Insertar un elemento:
s.insert(3); // O(logn)

// Ver cuantos elementos tiene: // O(1)
s.size()

// Eliminar un elemento determinado:
// Devuelve cuantos elementos borro
s.erase(1); // O(logn)

// Borra todos los elementos
us.clear();

// Verificar si un valor esta de forma rapido:
if (s.count(x)) { ... }

/*
Busca si un valor existe en el set.
Si no lo encuentra, devuelve s.end()
*/

s.find(3) // O(logn)

auto it = s.find(10);
if (it != s.end()) {
    cout << "Encontrado: " << *it << endl;
} else {
    cout << "No esta el 10\n";
}

// Otra especie de busqueda
auto it = s.lower_bound(25); // primer elemento >= 25
auto it2 = s.upper_bound(30); // primer elemento > 30

if (it != s.end()) cout << *it << endl;
if (it2 != s.end()) cout << *it2 << endl;
```

Listing 2.3: Funcionamiento del stack

Otras alternativas a set:

- `multiset`: Permite duplicados, ordenados también.
- `unordered_set`: Operaciones en $\mathcal{O}(1)$ promedio, pero sin orden.

2.4. Representación Binaria en C++

2.4.1. Mostrar un número en binario

La forma más directa es usando `bitset` de la STL:

```
int main() {  
    int x = 13;  
    cout << bitset<15>(x) << endl; // Muestra: 000000000001101  
}
```

Listing 2.4: Muestra un número en binario con 15 bits

2.4.2. Obtener la representación binaria como cadena

Podemos obtener la representación binaria como un `string`:

```
int main() {  
    int x = 13;  
    string bin = bitset<15>(x).to_string(); // "000000000001101"  
}
```

Listing 2.5: Convertir un número a cadena binaria

2.4.3. Conversión binaria manual

También podemos implementar la conversión binaria manualmente:

```
void mostrar_binario(int n) {  
    if (n == 0) {  
        cout << "0";  
        return;  
    }  
    string bin = "";  
    while (n > 0) {  
        bin = char('0' + (n % 2)) + bin;  
        n /= 2;  
    }  
    cout << bin << endl;  
}
```

Listing 2.6: Conversión manual de decimal a binario

Capítulo 3

Búsquedas

3.1. Búsqueda binaria

Siempre si vas a resolver algo con `lower_bound` aunque lo diga o no lo diga el problema SIEMPRE pero SIEMPRE ordenalo, me comi varios wa porque confiaba que siempre venia ordenado.

La función `lower_bound` es una herramienta fundamental en la biblioteca estándar de C++ para realizar búsquedas eficientes en contenedores ordenados.

3.1.1. Función `lower_bound`

- **Complejidad:** $O(\log n)$ para contenedores de acceso aleatorio como `vector`
- **Comportamiento:** Devuelve un iterador al primer elemento que **no es menor** que el valor dado (primer elemento $\geq$ al valor buscado)
- **Sintaxis:** `lower_bound(iterador_inicio, iterador_fin, valor)`

3.2. Problema de ejemplo

3.2.1. Descripción

El aula se representa como una línea unidimensional con celdas desde 1 hasta n . Inicialmente hay m profesores y David en posiciones distintas. En cada movimiento:

1. David se mueve a una celda adyacente o se queda
2. Todos los profesores se mueven simultáneamente a una celda adyacente o se quedan

David es atrapado cuando algún profesor está en la misma celda. Todos actúan óptimamente: David maximiza el número de movimientos hasta ser atrapado, mientras los profesores lo minimizan.

3.2.2. Solución

Para cada consulta con la posición a_i de David:

1. Encontrar los dos profesores más cercanos (izquierda y derecha)

2. Calcular la mitad de la distancia entre ellos
3. David se moverá al punto medio para maximizar el tiempo

3.2.3. Código de Solución

```

void solve() {
    ll n; cin >> n;
    ll m, q; cin >> m >> q;
    vi b(m);
    fore(i, 0, m) cin >> b[i];
    vi a(q);
    fore(i, 0, q) cin >> a[i];

    sort(all(b));

    fore(i, 0, q) {
        ll izq = 0, der = 0;
        ll res = 0;
        auto it = lower_bound(all(b), a[i]);

        if (it == b.begin()) {
            res = (*it) - 1;
        } else if (it == b.end()) {
            --it;
            res = n - (*it);
        } else {
            der = *it;
            --it;
            izq = *it;
            ll dist_der = der - a[i];
            ll dist_izq = a[i] - izq;
            res = (dist_der + dist_izq) / 2;
        }
        pri(res);
    }
}

```

Listing 3.1: Solución del problema

3.3. Uso de lower_bound para obtener índices

3.3.1. Obtención del índice

Cuando se utiliza `lower_bound` sobre un contenedor, podemos obtener la posición (índice) del elemento encontrado mediante aritmética de punteros:

```

void solve() {
    ll n; cin >> n; ll k; cin >> k;
    vi a(n); fore(i, 0, n) cin >> a[i];

    while(k--) {
        ll val; cin >> val;
        auto it = lower_bound(all(a), val);
    }
}

```

```

        ll res = it - a.begin(); // Obtenemos el indice
        pri(res + 1); // +1 para indice base 1
    }
}

```

Listing 3.2: Obtención del índice con lower_bound

3.3.2. Problema

Tenemos un edificio de n pisos, cada piso i con a_i departamentos. Las cartas están numeradas linealmente desde 1 hasta la suma total de departamentos. Dado un número de carta b_j , hay que determinar:

- El **piso** donde va la carta.
- El **departamento dentro del piso**.

Ejemplo: $n = 2$, $a = [3, 5]$, carta $b = 7 \rightarrow$ Piso 2, departamento 4.

Idea de la solución 1. Construimos un vector de **prefix sums**:

$$\text{pref}[0] = a_0, \quad \text{pref}[i] = \text{pref}[i-1] + a_i \text{ para } i \geq 1$$

Cada elemento indica la cantidad total de departamentos hasta ese piso.

2. Para cada carta b_j usamos **búsqueda binaria** con lower_bound:

$$\text{piso} = \text{lower_bound}(\text{pref.begin()}, \text{pref.end()}, b_j) - \text{pref.begin}()$$

3. Calculamos el departamento dentro del piso:

$$\text{depto} = b_j - (\text{piso} > 0 ? \text{pref}[\text{piso}-1] : 0)$$

—

Código en C++

```

void solve() {
    ll n, m;
    cin >> n >> m;
    vector<ll> a(n), b(m);
    for (int i = 0; i < n; i++) cin >> a[i];
    for (int i = 0; i < m; i++) cin >> b[i];

    // Construir prefix sums
    vector<ll> pref(n);
    pref[0] = a[0];
    for (int i = 1; i < n; i++) pref[i] = pref[i-1] + a[i];

    for (int i = 0; i < m; i++) {
        // Encontrar piso usando búsqueda binaria
        int piso = lower_bound(pref.begin(), pref.end(), b[i]) - pref.begin();
        // Departamento dentro del piso
        ll depto = b[i] - (piso > 0 ? pref[piso-1] : 0);
    }
}

```

```
cout << piso + 1 << " " << depto << "\n";
}
}
```

Ejemplo paso a paso

$$a = [3, 5, 2] \implies pref = [3, 8, 10], \quad b = [1, 4, 9]$$

Carta b_j	Piso (0-based)	Piso (1-based)	Departamento
1	0	1	1
4	1	2	1
9	2	3	1

Complejidad

- Construir prefix sums: $O(n)$
- Cada carta: $O(\log n)$ con `lower_bound`
- Total: $O(n + m \log n)$

Tips

- ‘`lower_bound`’ devuelve iterador, se resta ‘`pref.begin()`’ para obtener índice.
- Esta técnica combina **prefix sums** con **búsqueda binaria** para ubicar rápidamente rangos acumulados.

3.4. Búsqueda binaria en la respuesta

3.4.1. Caso general

```
bool can(int mid) {
    // Esta funcion debe responder si con mid como respuesta
    // es posible cumplir con la condicion del problema

    // Implementar la logica del chequeo

    return true;
}

int main(){
    // Busqueda binaria
    int low = 0, high = 1e9; // Ajustar limites segun el problema
    int ans = -1;

    while (low <= high) {
        int mid = (low + high) / 2;

        // si queremos el minimo que cumple
        if (check(mid)) {
            ans = mid;
            high = mid - 1;
        }
    }
}
```

```
    } else {  
        low = mid + 1;  
    }  
  
    // Si quieres el maximo que cumple:  
    if (check(mid)) {  
        ans = mid;  
        low = mid + 1;  
    } else {  
        high = mid - 1;  
    }  
}  
  
cout << ans << endl;  
}
```

Listing 3.3: Búsqueda binaria en la respuesta

3.4.2. Ejemplo 1: Problema de Sumas

```
ll n;  
bool can(ll x, vll &a) {  
    ll cont = 0; // Sup. disponibles  
    fore(i, 0, n) {  
        if (x < a[i]) return false;  
        cont += x - a[i];  
    }  
    return cont >= x;  
}  
  
void solve() {  
    cin >> n;  
    vll a(n);  
    fore(i, 0, n) cin >> a[i];  
  
    ll l = 0, r = 1e14, ans = 0;  
    while (l <= r) {  
        ll mid = (l + r) >> 1;  
        if (can(mid, a)) {  
            ans = mid;  
            r = mid - 1;  
        } else {  
            l = mid + 1;  
        }  
    }  
    pri(ans);  
}
```

Listing 3.4: Problema de sumas con búsqueda binaria

3.4.3. Ejemplo 2: Problema de Porcentajes

```
ll k, x, y, a, b, n;  
vll pr;
```

```

bool can(ll st) {
    vll p(st);
    for (int i = a - 1; i < st; i += a) p[i] += x;
    for (int i = b - 1; i < st; i += b) p[i] += y;
    sort(all(p), greater<ll>());
    ll cur = 0;
    fore(i, 0, st) cur += p[i] * pr[i] / 100;
    return cur >= k;
}

void solve() {
    pr.clear();
    cin >> n;
    fore(i, 0, n) {
        ll ai;
        cin >> ai;
        pr.pb(ai);
    }
    sort(all(pr), greater<int>());
    cin >> x >> a >> y >> b >> k;

    ll l = 0, h = n, ans = -1;
    while (l <= h) {
        ll mid = l + (h - l) / 2;
        if (can(mid)) {
            ans = mid;
            h = mid - 1;
        } else {
            l = mid + 1;
        }
    }
    pri(ans);
}

```

Listing 3.5: Problema de porcentajes con tickets

3.4.4. Ejemplo 3: Problema de Recursos

```

ll k, n;
vi a, b;

bool can(ll mid) {
    ll ka = k;
    fore(i, 0, n) {
        ll totact = a[i] * mid;
        if (totact > b[i]) {
            ll rest = totact - b[i];
            if (rest <= ka) {
                ka -= rest;
            } else {
                return false;
            }
        }
    }
    return true;
}

```

```
void solve() {
    cin >> n >> k;
    a.resize(n);
    b.resize(n);
    fore(i, 0, n) cin >> a[i];
    fore(i, 0, n) cin >> b[i];

    ll low = 0, high = 2e9 + 10;
    ll res = -1;

    while (low <= high) {
        ll mid = (low + high) / 2;
        if (can(mid)) {
            res = mid;
            low = mid + 1; // Buscamos el MAXIMO
        } else {
            high = mid - 1;
        }
    }
    pri(res);
}
```

Listing 3.6: Problema de gestión de recursos

3.4.5. Consideraciones Importantes

- **Límites:** Definir correctamente l y r según el problema
- **Función can:** Debe ser eficiente (generalmente $O(n)$ o $O(n \log n)$)
- **Overflow:** Usar $\text{mid} = l + (r - l) / 2$ para evitar overflow
- **Tipo de búsqueda:** Decidir si se busca el mínimo o máximo que cumple

3.4.6. Complejidad

La complejidad total es $O(T \times \log(\text{rango}) \times \text{complejidad de can})$, donde:

- T : número de casos de prueba
- **rango:** diferencia entre r y l
- **can:** complejidad de la función de verificación

Capítulo 4

Matemática

4.1. Operaciones basicas

4.1.1. Obtener π

```
const double PI = 3.141592653589793;  //FORMA 1
double pi = M_PI;                      //FORMA 2
```

Listing 4.1: Obtener π

4.1.2. Obtener la raiz cuadrada

Para calcular la raíz cuadrada de un número en C++, se utiliza la función `sqrt()`.

Ejemplo:

```
double x = 25.0;
double raiz = sqrt(x);
```

Notas importantes:

- El resultado puede guardarse en un `double` para conservar la precisión decimal.
- Si se guarda en un tipo entero, la parte decimal se truncará (se corta sin redondeo).

Complejidad: $O(\sqrt{n})$

```
bool esPrimo(int n) {
    if (n <= 1) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (int i = 3; i * i <= n; i += 2)
        if (n % i == 0)
            return false;
    return true;
}
```

Listing 4.2: Chequear si un número es primo

4.2. Reglas de Divisibilidad

Para determinar si un número es divisible por otro sin necesidad de hacer la división, se pueden usar las siguientes reglas de divisibilidad:

4.2.1. Reglas de Divisibilidad Básicas

1. **Divisible por 1:** Todos los números son divisibles por 1.
2. **Divisible por 2:** Un número es divisible por 2 si su último dígito es par (0, 2, 4, 6, 8).
3. **Divisible por 3:** Un número es divisible por 3 si la suma de sus dígitos es divisible por 3.
4. **Divisible por 4:** Un número es divisible por 4 si el número formado por sus últimos dos dígitos es divisible por 4.
5. **Divisible por 5:** Un número es divisible por 5 si su último dígito es 0 o 5.
6. **Divisible por 6:** Un número es divisible por 6 si es divisible por 2 y por 3.
7. **Divisible por 7:** Hay varias reglas:
 - **Regla 1:** Toma el último dígito, multiplícalo por 2 y réstalo del resto del número. Si el resultado es divisible por 7, entonces el número original también lo es.
 - **Regla 2:** Divide el número en bloques de 3 dígitos (de derecha a izquierda) y calcula la suma alternante de estos bloques. Si el resultado es divisible por 7, entonces el número original también lo es.
8. **Divisible por 8:** Un número es divisible por 8 si el número formado por sus últimos tres dígitos es divisible por 8.
9. **Divisible por 9:** Un número es divisible por 9 si la suma de sus dígitos es divisible por 9.
10. **Divisible por 10:** Un número es divisible por 10 si su último dígito es 0.

4.2.2. Reglas de Divisibilidad Menos Comunes

1. **Divisible por 11:**
 - **Regla 1:** Suma los dígitos en posiciones impares y resta la suma de los dígitos en posiciones pares. Si el resultado es divisible por 11, entonces el número original también lo es.
 - **Regla 2:** Alternativamente, puedes usar la suma alternante de los dígitos.
2. **Divisible por 12:** Un número es divisible por 12 si es divisible por 3 y por 4.
3. **Divisible por 13:**

- **Regla 1:** Toma el último dígito, multiplícalo por 9 y réstalo del resto del número. Si el resultado es divisible por 13, entonces el número original también lo es.
- **Regla 2:** Divide el número en bloques de 3 dígitos y calcula la suma alternante de estos bloques. Si el resultado es divisible por 13, entonces el número original también lo es.

4. Divisible por 17:

- **Regla 1:** Toma el último dígito, multiplícalo por 5 y réstalo del resto del número. Si el resultado es divisible por 17, entonces el número original también lo es.

5. Divisible por 19:

- **Regla 1:** Toma el último dígito, multiplícalo por 2 y súmalo al resto del número. Si el resultado es divisible por 19, entonces el número original también lo es.

4.2.3. Reglas de Divisibilidad para Números Primos Mayores

Para números primos mayores, como 23, 29, etc., las reglas pueden volverse más complejas y a menudo implican métodos similares a los usados para 7, 13, 17 y 19, pero con diferentes multiplicadores.

4.2.4. Métodos Generales para Verificar Divisibilidad

1. **División Directa:** La forma más directa de verificar la divisibilidad es realizar la división y ver si el residuo es cero.
2. **Uso de Factores:** Si un número es divisible por dos o más números coprimos, entonces es divisible por su producto. Por ejemplo, si un número es divisible por 3 y por 5, entonces es divisible por 15.

4.2.5. Generación de divisores

Para encontrar todos los divisores de un número k :

```
for (ll i = 1; i * i <= k; i++) {
    if (k % i == 0) {
        divs.push_back(i);
        if (i != k / i) divs.push_back(k / i);
    }
}
```

Listing 4.3: Generar los divisores de un numero

4.3. Números Primos

4.3.1. Chequear si un número es primo

Complejidad: $O(\sqrt{n})$

```

bool esPrimo(int n) {
    if (n <= 1) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (int i = 3; i * i <= n; i += 2)
        if (n % i == 0)
            return false;
    return true;
}

```

Listing 4.4: Chequear si un número es primo

4.3.2. Criba de Eratóstenes para factorizar números

Uso

Guarda para cada número su **menor factor primo** (SPF, *smallest prime factor*). Con eso, puedes factorizar cualquier número $x \leq n$ muy rápido.

```

vector<int> spf; // spf[x] = menor factor primo de x

void build_spf(int n) {
    spf.assign(n + 1, 0);
    for (int i = 2; i <= n; i++) {
        if (spf[i] == 0) {
            spf[i] = i; // i es primo
            if (1LL * i * i <= n) {
                for (long long j = 1LL * i * i; j <= n; j += i) {
                    if (spf[j] == 0) spf[j] = i;
                }
            }
        }
    }
}

vector<pair<int,int>> factorize(int x) {
    vector<pair<int,int>> fac;
    while (x > 1) {
        int p = spf[x], e = 0;
        while (x % p == 0) {
            x /= p;
            e++;
        }
        fac.push_back({p, e});
    }
    return fac;
}

int main() {
    int n = 100; // limite maximo que vas a factorizar
    build_spf(n);
}

```

```

int x = 6;
auto f = factorize(x);

// salida: 6 = 2^1 * 3^1
cout << x << " = ";
for (int i = 0; i < (int)f.size(); i++) {
    cout << f[i].first << "^" << f[i].second;
    if (i + 1 < (int)f.size()) cout << " * ";
}
}

```

Listing 4.5: Criba SPF + factorización en primos

Complejidad:

- Preprocesamiento de la criba SPF hasta n : $O(n \log \log n)$.
- Memoria: $O(n)$.
- Factorizar un número x : $O(k)$, donde k es la cantidad de factores primos (con multiplicidad). En práctica: muy rápido.

4.4. Permutaciones

4.4.1. Computar las permutaciones

Sea un arreglo $A = [a_1, a_2, \dots, a_n]$. Queremos generar todas las permutaciones de sus elementos.

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> A = {1, 2, 3};
    sort(A.begin(), A.end()); // ordenar antes de usar next_permutation

    do {
        for (int x : A) cout << x << " ";
        cout << "\n";
    } while (next_permutation(A.begin(), A.end()));

    return 0;
}

```

Listing 4.6: Chequear si un número es primo

4.5. Geometría

4.5.1. Distancia entre dos puntos

Dados dos puntos (x_1, y_1) y (x_2, y_2) , la distancia euclidiana es:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

En programación competitiva, muchas veces conviene usar la distancia al cuadrado (para evitar `sqrt`):

$$d^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$$

```
11 dist2 = (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2);
```

Listing 4.7: Distancia al cuadrado entre dos puntos

4.5.2. Triángulos

Un triángulo queda determinado si conocés:

- 3 lados (SSS)
- 2 lados + 1 ángulo (SAS)
- 2 ángulos + 1 lado (ASA / AAS)

Ley de Cosenos

Se usa cuando tenés lados y querés ángulos, o viceversa:

$$c^2 = a^2 + b^2 - 2ab \cos(C)$$

Sirve para:

- SSS → sacar ángulos
- SAS → sacar el lado faltante

Ejemplo (CP):

```
double C = acos((a * a + b * b - c * c) / (2.0 * a * b));
```

Listing 4.8: Ley de Cosenos: obtener un ángulo

Ley de Senos

Cuando tenés ángulos y lados mezclados:

$$\frac{\sin(A)}{a} = \frac{\sin(B)}{b} = \frac{\sin(C)}{c}$$

Sirve para:

- ASA / AAS → sacar lados
- completar triángulos

Ejemplo:

```
double b = a * sin(B) / sin(A);
```

Listing 4.9: Ley de Senos: obtener un lado

Capítulo 5

Programacion Dinamica (dp)

5.1. De una función recursiva por partes a código con DP

En muchos problemas, partimos de una definición de función recursiva por partes. El objetivo es transformarla en un algoritmo eficiente usando programación dinámica. A continuación se muestra un ejemplo general del proceso.

Ejemplo: función definida por partes

Supongamos que tenemos la siguiente definición matemática:

$$f(i) = \begin{cases} 0 & \text{si } i = 0 \\ f(i-1) + |a[i] - a[i-1]| & \text{si } i = 1 \\ \min(f(i-1) + |a[i] - a[i-1]|, f(i-2) + |a[i] - a[i-2]|) & \text{si } i \geq 2 \end{cases}$$

Queremos implementarla eficientemente para valores grandes de i .

Paso 1: Identificar el estado

El subproblema está determinado solo por el índice i . Por lo tanto, usamos un arreglo $dp[i]$ donde $dp[i]$ almacena el valor de $f(i)$.

Paso 2: Código en C++ con memoización

```
const int MAXN = 1e5+10;
int dp[MAXN];
int a[MAXN];

int f(int i) {
    if (dp[i] != -1) return dp[i];

    if (i == 0) return dp[i] = 0;
    if (i == 1) return dp[i] = f(0) + abs(a[1] - a[0]);
```

```

return dp[i] = min(
f(i - 1) + abs(a[i] - a[i - 1]),
f(i - 2) + abs(a[i] - a[i - 2])
);
}

```

Listing 5.1: Función recursiva con memoización

Inicialización:

```
fill(dp, dp + n, -1);
```

Llamada inicial:

```
cout << f(n - 1) << '\n';
```

Paso 3: Versión iterativa (tabulación)

```

dp[0] = 0;
dp[1] = abs(a[1] - a[0]);

for (int i = 2; i < n; ++i) {
dp[i] = min(
dp[i - 1] + abs(a[i] - a[i - 1]),
dp[i - 2] + abs(a[i] - a[i - 2])
);
}

```

Listing 5.2: Versión iterativa (bottom-up)

Resumen

- La clave está en identificar bien los **casos base**.
- Luego traducir cada parte de la definición en código con condiciones explícitas.
- Asegurate de guardar los resultados para no recalcular.

5.2. Ejemplos clásicos**5.2.1. A - Frog 1**

Problema: dado un arreglo de alturas, calcular el costo mínimo para saltar desde el índice 0 al último, con saltos de 1 o 2 posiciones.

```

11 dp[MAXN];
11 a[MAXN];
11 f(int i) {
11 if (i == 0) return 0;
11 if (dp[i] != -1) return dp[i];
11 res = f(i-1) + abs(a[i] - a[i-1]);
11 if (i > 1) res = min(res, f(i-2) + abs(a[i] - a[i-2]));

```

```
return dp[i] = res;
}
```

Listing 5.3: Frog1: DP mínima energía

Estados: posición actual. Transiciones: desde ‘i-1’ o ‘i-2’. Caso base: ‘dp[0] = 0’.

5.2.2. C - Vacation

Problema: en cada día, podés hacer una de 3 actividades con recompensa. No podés repetir la misma actividad dos días seguidos.

```
11 dp[MAXN][3];
11 a[MAXN][3];

11 f(int i, int last) {
11     if (i == n) return 0;
11     if (dp[i][last] != -1) return dp[i][last];
11     res = 0;
11     for(j, 0, 3) {
11         if (j != last) {
11             res = max(res, a[i][j] + f(i+1, j));
11         }
11     }
11     return dp[i][last] = res;
11 }
```

Listing 5.4: Vacation: DP sobre días y última actividad

Estado: día actual y actividad anterior. Transiciones: elegir una actividad distinta a la del día anterior. Caso base: cuando ‘i == n’.

5.2.3. D - Knapsack 0/1

Problema clásico: elegir un subconjunto de objetos sin pasar un peso máximo.

```
11 dp[MAXW];
11 int w[MAXN], v[MAXN];

11 void solve() {
11     for (int i = 0; i < n; ++i) {
11         for (int j = W; j >= w[i]; --j) {
11             dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
11         }
11     }
11 }
```

Listing 5.5: Mochila 0/1: DP por peso

Estado: peso total actual. Transiciones: elegir o no cada objeto. Usa optimización de espacio (1D).

5.2.4. G - Longest Path in DAG

Problema: encontrar el camino más largo en un grafo dirigido acíclico.

```
vector<int> G[MAXN];
int dp[MAXN];

int dfs(int u) {
    if (dp[u] != -1) return dp[u];
    int res = 0;
    for (int v : G[u]) {
        res = max(res, 1 + dfs(v));
    }
    return dp[u] = res;
}
```

Listing 5.6: Camino más largo en DAG con DP + DFS

Estado: nodo actual. Transiciones: recorrer vecinos. Requiere topological sort o memoización con DFS.

5.3. Resumen

- Pensar en el subproblema mínimo que necesitás.
- Formular la recurrencia primero.
- Escribir la versión recursiva con memoización es buena forma de empezar.
- Luego podés optimizar a iterativa si hace falta.

Capítulo 6

Grafos

6.1. Representaciones comunes

Lista de adyacencia

```
vector<int> G[MAXN]; // grafo sin peso

void agregar_arista(int u, int v) {
    G[u].push_back(v);
    G[v].push_back(u); // si es no dirigido
}
```

Listing 6.1: Lista de adyacencia

Matriz de adyacencia

```
bool mat[MAXN][MAXN];

void agregar_arista(int u, int v) {
    mat[u][v] = true;
    mat[v][u] = true; // si es no dirigido
}
```

Listing 6.2: Matriz de adyacencia

6.2. Recorridos clásicos

6.2.1. DFS (Depth-First Search)

```
bool vis[MAXN];
vector<int> G[MAXN];

void dfs(int u) {
    vis[u] = true;
    for (int v : G[u]) {
        if (!vis[v]) dfs(v);
    }
}
```

```
    }  
}
```

Listing 6.3: DFS no dirigido

6.2.2. BFS (Breadth-First Search)

```
int dist[MAXN];  
bool vis[MAXN];  
vector<int> G[MAXN];  
  
void bfs(int s) {  
    queue<int> q;  
    q.push(s);  
    vis[s] = true;  
    dist[s] = 0;  
  
    while (!q.empty()) {  
        int u = q.front(); q.pop();  
        for (int v : G[u]) {  
            if (!vis[v]) {  
                vis[v] = true;  
                dist[v] = dist[u] + 1;  
                q.push(v);  
            }  
        }  
    }  
}
```

Listing 6.4: BFS con cálculo de distancias

6.3. Componentes conexas

```
int comp = 0;  
  
for (int i = 0; i < n; ++i) {  
    if (!vis[i]) {  
        dfs(i);  
        comp++;  
    }  
}
```

Listing 6.5: Contar componentes conexas

6.4. Dijkstra (caminos mínimos)

```

typedef pair<int, int> pii;
vector<pii> G[MAXN]; // {peso, destino}
int dist[MAXN];

void dijkstra(int s) {
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    fill(dist, dist + MAXN, INF);
    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [w, v] : G[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}

```

Listing 6.6: Dijkstra con priority queue

6.5. Bellman-Ford

```

vector<tuple<int, int, int>> edges; // {u, v, peso}
int dist[MAXN];

bool bellman(int s, int n) {
    fill(dist, dist + MAXN, INF);
    dist[s] = 0;

    for (int i = 0; i < n - 1; ++i) {
        for (auto [u, v, w] : edges) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }

    // Detectar ciclos negativos
    for (auto [u, v, w] : edges) {
        if (dist[u] + w < dist[v]) return true;
    }

    return false;
}

```

Listing 6.7: Bellman-Ford para grafos con pesos negativos

6.6. Problemas clásicos

A - Número de componentes conexas

Estado: nodos visitados. **Recorrido:** DFS o BFS. **Complejidad:** $O(n + m)$

B - Camino más corto (no ponderado)

Idea: usar BFS desde el nodo origen y guardar las distancias.

C - Dijkstra / Bellman-Ford

Cuando usar cada uno:

- Dijkstra $\rightarrow$ aristas no negativas. - Bellman-Ford $\rightarrow$ permite pesos negativos.

6.7. ¿Cómo verificar si un grafo es una línea (path)?

1. Verificar que $M = N - 1$ (condición necesaria para ser un árbol).
2. Verificar que el grafo es conexo (DFS o BFS desde cualquier nodo).
3. Contar los grados de los nodos:
 - Debe haber exactamente 2 nodos con grado 1.
 - Todos los demás deben tener grado 2.

Código en C++

```
vector<int> G[MAXN];
bool vis[MAXN];
int deg[MAXN];

void dfs(int u) {
    vis[u] = true;
    for (int v : G[u]) {
        if (!vis[v]) dfs(v);
    }
}

bool es_linea(int n, int m) {
    if (m != n - 1) return false; // debe tener N-1 aristas

    // verificar conectividad
    fill(vis, vis + n, false);
    dfs(0);
    for (int i = 0; i < n; ++i)
        if (!vis[i]) return false;

    // contar grados
    int cnt_deg1 = 0;
```

```

    for (int i = 0; i < n; ++i) {
        if (deg[i] == 1) cnt_deg1++;
        else if (deg[i] != 2) return false;
    }

    return cnt_deg1 == 2;
}

```

Listing 6.8: Verificar si un grafo es una línea

Entrada:

- Llenar $G[i]$ con las aristas del grafo.
- Aumentar $\deg[u]$ y $\deg[v]$ cada vez que se agrega una arista entre u y v .

6.8. Detectar ciclos en grafo no dirigido

1. Usar DFS manteniendo el nodo padre
2. Si encuentras un nodo ya visitado que no es el padre, hay ciclo

Código en C++

```

vector<int> G[MAXN];
bool vis[MAXN];

bool hasCycle(int u, int parent) {
    vis[u] = true;
    for (int v : G[u]) {
        if (!vis[v]) {
            if (hasCycle(v, u))
                return true;
        }
        else if (v != parent)
            return true;
    }
    return false;
}

bool hasCycleAll(int n) {
    memset(vis, 0, sizeof(vis));
    for (int i = 0; i < n; ++i) {
        if (!vis[i]) {
            if (hasCycle(i, -1)) {
                return true;
            }
        }
    }
    return false;
}

```

Listing 6.9: Detectar ciclos en grafo no dirigido

6.9. Detectar ciclos en grafo dirigido

Inicialmente, todos los nodos están en blanco (0), lo que significa que no fueron visitados. Cuando empezamos a explorar un nodo, lo marcamos como gris (1), indicando que está en proceso dentro de la recursión. Luego recorremos sus vecinos: si encontramos uno blanco, seguimos el DFS; pero si encontramos un nodo gris, significa que volvimos a un nodo que todavía está en la pila de recursión, lo que implica la existencia de un ciclo (una arista de retroceso o back edge). Finalmente, cuando terminamos de procesar un nodo, lo marcamos como negro (2), indicando que ya fue completamente explorado. Para asegurarnos de detectar ciclos en todo el grafo, este proceso se inicia desde cada nodo no visitado. La función `hasCycleAll()` es la que se usaría en el `main` para verificar que tu grafo tiene un ciclo.

1. Usar DFS con colores (0: no visitado, 1: en proceso, 2: visitado)
2. Si encuentras un nodo .en proceso" durante el DFS, hay ciclo

Código en C++

```
vector<int> G[MAXN];  
// 0: blanco (no visitado)  
// 1: gris (en proceso)  
// 2: negro (visitado)  
int color[MAXN];  
  
bool hasCycleDirected(int u) {  
    color[u] = 1; // gris  
    for (int v : G[u]) {  
        if (color[v] == 0) {  
            if (hasCycleDirected(v))  
                return true;  
        }  
        else if (color[v] == 1)  
            return true;  
    }  
    color[u] = 2; // negro  
    return false;  
}  
  
bool hasCycleAll(int n) {  
    fill(color, color + n, 0);  
    for (int i = 0; i < n; ++i) {  
        if (color[i] == 0 && hasCycleDirected(i))  
            return true;  
    }  
    return false;  
}
```

}

Listing 6.10: Detectar ciclos en grafo dirigido

6.10. Verificar si hay ciclo impar

1. Intentar colorear el grafo con 2 colores (bipartición)
2. Si no es posible, contiene ciclo impar

Código en C++

```
vector<int> G[MAXN];
int col[MAXN]; // 0 o 1

bool tieneCicloImpar(int n) {
    queue<int> q;
    fill(col, col + n, -1);

    col[0] = 0;
    q.push(0);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : G[u]) {
            if (col[v] == -1) {
                col[v] = 1 - col[u];
                q.push(v);
            }
            else if (col[v] == col[u]) {
                return true; // Ciclo impar encontrado
            }
        }
    }
    return false;
}
```

Listing 6.11: Detectar ciclo impar

6.11. Verificar si el grafo es bipartito

1. Igual que detectar ciclo impar, pero devolviendo el resultado inverso
2. Si se puede colorear con 2 colores, es bipartito

Código en C++

```

vector<int> G[MAXN];
int col[MAXN]; // 0 o 1

bool esBipartito(int n) {
    queue<int> q;
    fill(col, col + n, -1);

    col[0] = 0;
    q.push(0);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : G[u]) {
            if (col[v] == -1) {
                col[v] = 1 - col[u];
                q.push(v);
            }
            else if (col[v] == col[u]) {
                return false;
            }
        }
    }
    return true;
}

```

Listing 6.12: Verificar si grafo es bipartito

6.12. Distancia mínima desde un nodo

1. Usar BFS para calcular distancias desde el nodo fuente
2. Mantener arreglo de distancias inicializado en -1

Código en C++

```

vector<int> G[MAXN];
int dist[MAXN];

void bfs(int s, int n) {
    queue<int> q;
    fill(dist, dist + n, -1);

    dist[s] = 0;
    q.push(s);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : G[u]) {
            if (dist[v] == -1) {

```

```

        dist[v] = dist[u] + 1;
        q.push(v);
    }
}
}
}

```

Listing 6.13: Distancia mínima con BFS

6.13. Verificar si el grafo es un árbol

1. Verificar que tiene exactamente $N-1$ aristas
2. Verificar que es conexo
3. Verificar que no tiene ciclos

Código en C++

```

vector<int> G[MAXN];
bool vis[MAXN];

bool hasCycle(int u, int parent) {
    vis[u] = true;
    for (int v : G[u]) {
        if (!vis[v]) {
            if (hasCycle(v, u)) return true;
        }
        else if (v != parent) return true;
    }
    return false;
}

bool esArbol(int n, int m) {
    if (m != n - 1) return false;

    // Verificar conexidad
    fill(vis, vis + n, false);
    hasCycle(0, -1);
    for (int i = 0; i < n; ++i)
        if (!vis[i]) return false;

    // Verificar que no tiene ciclos
    fill(vis, vis + n, false);
    if (hasCycle(0, -1)) return false;

    return true;
}

```

Listing 6.14: Verificar si grafo es árbol

6.14. Número de hojas en un árbol

1. Contar nodos con grado 1 (excepto posiblemente la raíz en árboles enraizados)

Código en C++

```
vector<int> G[MAXN];
int deg[MAXN];

int contarHojas(int n) {
    int hojas = 0;
    for (int i = 0; i < n; ++i) {
        if (G[i].size() == 1) // grado 1
            hojas++;
    }
    return hojas;
}
```

Listing 6.15: Contar hojas en un árbol

6.15. Reconstrucción de Caminos (BFS)

Cuando necesitas no solo la distancia mínima, sino la secuencia de nodos, se utiliza un arreglo de predecesores (o padres).

1. Concepto Clave: El arreglo parent

- `parent[v] = u`: Significa que llegamos al nodo `v` desde el nodo `u`.
- Inicializamos todo el arreglo en -1 (o un valor nulo) para identificar el origen y los nodos no visitados.

2. Implementación del BFS

```
// n = numero de nodos, start = nodo inicial
vector<int> parent(n + 1, -1);
vector<int> dist(n + 1, -1);
queue<int> q;

q.push(start);
dist[start] = 0;

while(!q.empty()){
    int u = q.front(); q.pop();

    for(int v : G[u]){
        if(dist[v] == -1){ // Si no ha sido visitado
            dist[v] = dist[u] + 1;
            parent[v] = u; // REGISTRO: u es el padre de v
        }
    }
}
```

```

        q.push(v);
    }
}

```

Listing 6.16: BFS con arreglo parent

3. Función para recuperar el camino

Una vez termina el BFS, el camino está encadenado desde el destino hacia atrás. Debes seguir los punteros hasta llegar al origen y luego invertir la lista.

```

vector<int> getPath(int start, int end, const vector<int>& parent) {
    vector<int> path;

    // Si el destino no tiene padre y no es el inicio, no hay camino
    if (parent[end] == -1 && start != end) return {};

    // Retrocedemos desde 'end' hasta 'start'
    for (int v = end; v != -1; v = parent[v]) {
        path.push_back(v);
    }

    // El camino esta [destino, ..., origen], lo invertimos
    reverse(path.begin(), path.end());

    return path;
}

```

Listing 6.17: Función para reconstruir el camino

6.16. Orden Topológico

Se te proporciona un grafo dirigido con n vértices y m aristas. Tienes que encontrar un orden de los vértices, de modo que cada arista vaya desde el vértice con un índice menor hasta un vértice con un índice mayor.

En otras palabras, se trata de encontrar una permutación de los vértices (orden topológico) que corresponda al orden definido por todas las aristas del grafo. Un orden topológico puede no existir en absoluto. Solo existe si el grafo dirigido no contiene ciclos (DAG).

Ejemplo de uso

Un problema común en el que se produce la ordenación topológica es el siguiente. Hay n variables con valores desconocidos. Para algunas variables, sabemos que una de ellas es menor que la otra. Debes comprobar si estas restricciones son contradictorias y, si no lo son, mostrar las variables en orden ascendente (si hay varias respuestas posibles, mostrar cualquiera de ellas). Es fácil notar que este es exactamente el problema de encontrar el orden topológico de un grafo con n vértices.

¿Cómo funciona con DFS?

Supongamos que el grafo es acíclico. ¿Qué hace la búsqueda en profundidad (DFS)?

Al comenzar desde algún vértice v , DFS intenta recorrer a lo largo de todas las aristas que salen de v . Se detiene en los bordes cuyos extremos ya han sido visitados previamente, y recorre el resto de los bordes y continúa recursivamente en sus extremos.

Por lo tanto, en el momento en que la llamada a la función $\text{dfs}(v)$ ha terminado, todos los vértices que son alcanzables desde v han sido visitados por la búsqueda, ya sea directamente (a través de una arista) o indirectamente.

Añadamos el vértice v a una lista cuando terminemos $\text{dfs}(v)$. Como todos los vértices alcanzables ya han sido visitados, ya estarán en la lista cuando agreguemos v . Hagamos esto para cada vértice del grafo, con una o varias búsquedas en profundidad. Para cada arista dirigida $v \rightarrow u$ en el gráfico, u aparecerá antes en esta lista que v , porque u es accesible desde v .

Entonces, si simplemente etiquetamos los vértices en esta lista con $n-1, n-2, \dots, 1, 0$, hemos encontrado un orden topológico del grafo. En otras palabras, la lista representa el orden topológico inverso, por lo que basta con invertir el arreglo al final.

Código en C++

Aquí se presenta una implementación que asume que el grafo es acíclico, es decir, que existe el orden topológico deseado:

```
// n = numero de vertices
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u]) {
            dfs(u);
        }
    }
    ans.push_back(v);
}

void topological_sort() {
    visited.assign(n, false);
    ans.clear();
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            dfs(i);
        }
    }
    reverse(ans.begin(), ans.end());
}
```

Listing 6.18: Orden Topológico con DFS

La función principal de la solución es `topological_sort`, que inicializa las variables DFS, lanza DFS y recibe la respuesta en el vector `ans`.

Vale la pena señalar que cuando el grafo **no** es acíclico, el resultado de `topological_sort` aún sería algo significativo en el sentido de que si un vértice u es alcanzable desde el vértice v , pero no al revés, el vértice v siempre aparecerá primero en el array resultante.

Capítulo 7

Strings

7.1. Permutaciones de string

```
int main() {
    string s = "ABA";

    IMPORTANTE: Ordenar la cadena primero
    sort(s.begin(), s.end());

    El bucle do-while asegura que se procese la 1 permutacion
    do {
        cout << s << endl;
    } while (next_permutation(s.begin(), s.end()));

    return 0;
}
```

Listing 7.1: Next_permutation

Complejidad: Para una cadena de longitud n es $n!$.

Capítulo 8

Estructuras de Datos

8.1. Union Find / Disjoint Set Union (DSU)

El **Union-Find** es una estructura de datos que permite mantener un conjunto de elementos particionados en subconjuntos disjuntos. Es muy eficiente para operaciones de conectividad, ya que nos dice si dos elementos pertenecen al mismo grupo y nos permite unirlos. Se basa en dos operaciones principales:

- **find(x)**: Sirve para saber a qué conjunto pertenece el elemento x . Devuelve el representante" (o raíz) de ese conjunto.
- **unite(a, b)** o **union(a, b)**: Une los conjuntos a los que pertenecen a y b .

Uso en grillas 2D

El Union-Find está diseñado para trabajar con estructuras en una sola dimensión (1D), es decir, maneja elementos como si fueran posiciones en un arreglo lineal $(0, 1, 2, \dots, N - 1)$. Por eso, cuando el problema es en 2D, como una grilla de filas y columnas (r, c) , primero hay que convertir cada celda en un único número para poder usar DSU correctamente.

Esto se hace con la siguiente fórmula:

$$id = r \times m + c$$

(Donde m es la cantidad de columnas de la grilla).

```
struct DSU {
    vector<int> parent;
    vector<int> sz; // Tamano del componente

    DSU(int n) {
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        for (int i = 0; i <= n; i++) parent[i] = i;
    }

    // Encuentra el representante con Compresion de Caminos
    int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]);
    }
}
```

```

// Une dos conjuntos por Tamano
void unite(int i, int j) {
    int root_i = find(i);
    int root_j = find(j);
    if (root_i != root_j) {
        if (sz[root_i] < sz[root_j]) swap(root_i, root_j);
        parent[root_j] = root_i;
        sz[root_i] += sz[root_j];
    }
}

bool same(int i, int j) {
    return find(i) == find(j);
}
};

```

8.2. Fenwick Tree (BIT)

El **Fenwick Tree** o **Binary Indexed Tree (BIT)** es una estructura de datos sobre un arreglo de números que permite mantener información acumulada y responder consultas de forma eficiente.

Se usa principalmente para dos operaciones:

- **Actualización de punto:** sumar un valor a una posición específica.
- **Consulta de prefijo:** obtener la suma de los elementos desde la posición 1 hasta i .

Complejidad temporal

- update: $O(\log n)$
- query: $O(\log n)$

Complejidad espacial: $O(n)$.

Idea principal

Cada posición del BIT guarda la suma de un bloque de elementos del arreglo original. Gracias a esto, para actualizar o consultar no es necesario recorrer todo el arreglo: solo se visitan algunos nodos relacionados con el bit menos significativo de cada índice.

Función `update(idx, val)`

Sirve para sumar `val` en la posición `idx`.

En la práctica, el arreglo del BIT y el tamaño `n` suelen declararse como variables globales o como atributos de una estructura/clase, porque las funciones `update` y `query` necesitan acceder a ellos. No es obligatorio que sean globales: también se pueden pasar como parámetros o encapsular dentro de un `struct`, pero en programación competitiva se usa mucho la versión global por simplicidad.

- **Cómo funciona:** empieza en `idx` y va subiendo hacia sus "padres" en la estructura, agregando el valor en cada bloque afectado.
- **Importante:** este BIT normalmente trabaja con índices desde 1 hasta n .

```
void update(int idx, int val) {
    for (; idx <= n; idx += idx & -idx)
        bit[idx] += val;
}
```

Función query(idx)

Calcula la suma de los elementos desde la posición 1 hasta `idx`.

- **Cómo funciona:** suma el valor en `idx` y luego retrocede eliminando el bit menos significativo hasta llegar a cero.

```
int query(int pos) {
    int res = 0;
    for (; pos > 0; pos -= pos & -pos)
        res += bit[pos];
    return res;
}
```

Para obtener la suma de un rango $[L, R]$, se usa:

$$query(R) - query(L - 1)$$

¿En qué tipos de problemas sirve?

El BIT es útil cuando necesitas una mezcla de consultas y modificaciones en tiempo real, sobre todo si ambas afectan un solo punto del arreglo.

- **Cálculo de inversiones:** contar pares (i, j) tales que $i < j$ y $A[i] > A[j]$.
- **Actualizaciones de punto y consultas de rango:** cuando el arreglo cambia constantemente y necesitas responder sumas rápidas.
- **Conteo de frecuencias:** mantener cuántas veces aparece cada valor para responder consultas sobre prefijos o rangos de frecuencias.

Observación

Si el problema requiere actualizaciones en un rango completo o consultas más complejas, a veces conviene usar un segment tree; si solo necesitas sumas de prefijos con cambios puntuales, el Fenwick Tree suele ser más simple y rápido de implementar.

Capítulo 9

Arrays / Subarrays

9.1. Kadane

Kadane es un algoritmo para encontrar el subarreglo contiguo con mejor suma en tiempo lineal.

¿Qué hace? Recorre el arreglo una sola vez y mantiene:

- **cur**: la mejor suma que termina en la posición actual
- **best**: la mejor suma global encontrada hasta el momento

Para cada elemento **x**, decide entre:

- empezar un subarreglo nuevo en **x**
- extender el subarreglo anterior (**cur + x**)

```
long long kadane_max(const vector<long long>& arr) {
    long long cur = arr[0], best = arr[0];
    for (int i = 1; i < (int)arr.size(); ++i) {
        cur = max(arr[i], cur + arr[i]);
        best = max(best, cur);
    }
    return best;
}
```

Listing 9.1: Kadane para subarreglo de suma maxima

Cómo usarlo para buscar el mínimo La idea es la misma, pero reemplazando max por min.

Complejidad Tanto **kadane_max** como **kadane_min** tienen:

- **Tiempo**: $O(n)$
- **Memoria extra**: $O(1)$

9.1.1. Kadane con restricción (el subarreglo debe incluir **k**)

Si queremos la suma máxima de un subarreglo contiguo que **obligatoriamente** contenga la posición **k**, podemos precomputar:

- **left[i]**: mejor suma de un subarreglo que termina en **i**

- `right[i]`: mejor suma de un subarreglo que empieza en `i`

Entonces, la mejor respuesta que incluye `k` es:

$$\text{left}[k] + \text{right}[k] - a[k]$$

```
long long kadane_con_k_max(const vector<long long>& a, int k) {
    int n = (int)a.size();
    vector<long long> left(n), right(n);

    left[0] = a[0];
    for (int i = 1; i < n; i++) {
        left[i] = max(a[i], left[i - 1] + a[i]);
    }

    right[n - 1] = a[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        right[i] = max(a[i], right[i + 1] + a[i]);
    }

    return left[k] + right[k] - a[k];
}
```

Listing 9.2: Kadane maximo forzando incluir la posicion `k` (0-based)

Complejidad de esta variante

- **Tiempo:** $O(n)$
- **Memoria extra:** $O(n)$ por los arreglos `left` y `right`